



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

SDS III: State



Clock and Signals

- Clock and Signals
- Registers
- Register Timing
- Circuit Timing Constraints
- Finite State Machines



Synchronous Digital System (SDS)

[review]

The hardware underlying almost every processor is a **Synchronous Digital System**.

- **Synchronous**: All operations coordinated by a central **clock**.
 - "Heartbeat" of the system!
- **Digital**: Represent all values by discrete values, i.e., **binary digits**.
 - We've seen how to represent many symbols via 1s and 0s.
This representation extends to electrical signals!

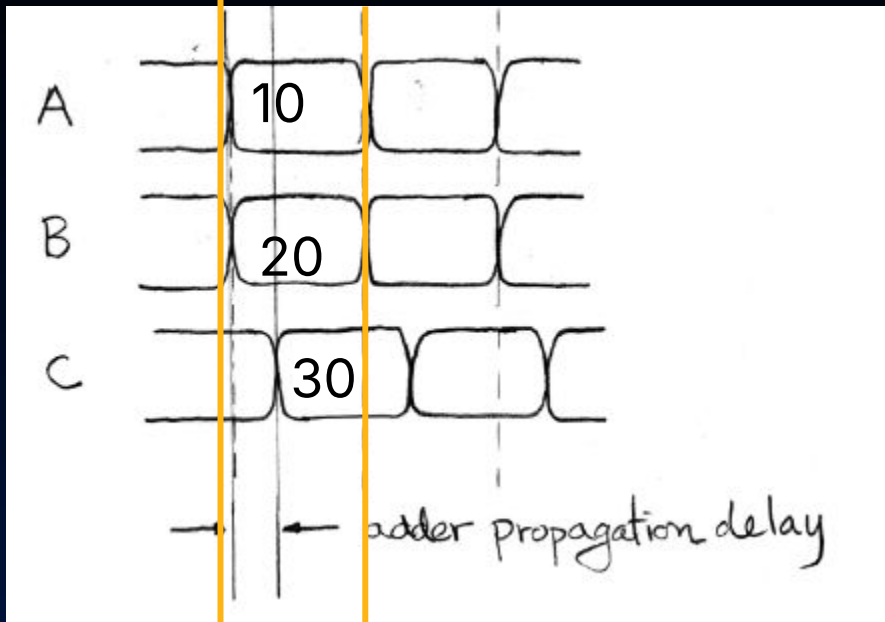
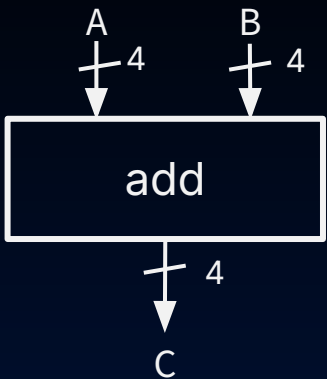


The Clock

- The clock controls flow of information, i.e., updates triggers with a clock signal.
 - Typical **clock period**:
 $T = 1\text{ns} = 10^{-9}\text{ s}$,
 - **Clock frequency**:
 $f = 1/T = 1\text{ GHz} = 1\text{ billion cycles/sec}$



Propagation Delay: Combinational Logic Blocks



All combinational logic blocks have **propagation delay**: time taken for electrical charge to move from input, through wires/transistors, to output

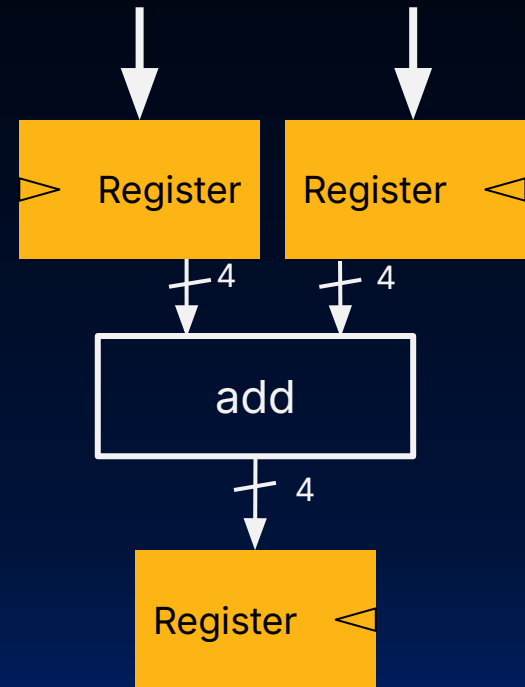


Registers

- Clock and Signals
- Registers
- Register Timing
- Circuit Timing Constraints
- Finite State Machines

Introducing State Elements!

- **State Elements** are circuits that store information.
 - Example: **Registers**
 - Another example: caches, main memory
- State Elements control. flow of information between combinational logic blocks
 - Used to hold up the movement of information at inputs to combinational logic blocks, allow for orderly passage





Registers

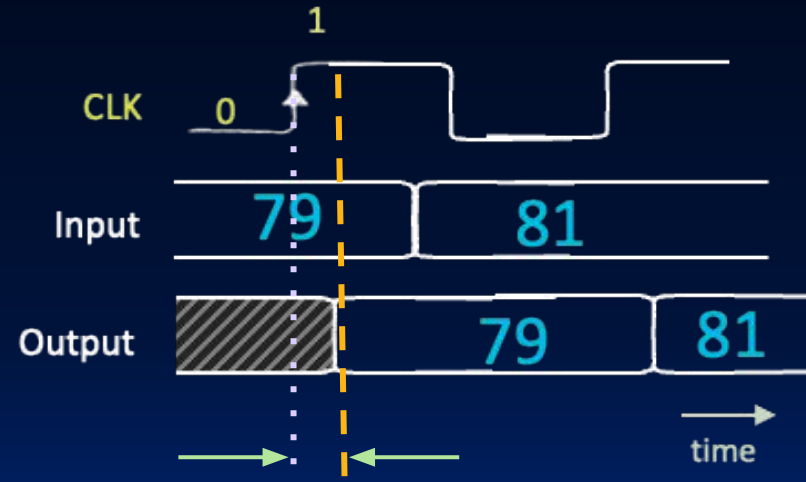
- A **register** is a special circuit that "stores information" for a period of time.
 - LOAD "control signal" is a trigger.
 - On trigger: Sample Input, then transfer to Output.
 - At all other times, ignore the Input.





Clocked Registers

- **Clocked, rising-edge triggered** register:
 - Trigger: LOAD signal goes from 0 to 1
 - Connect system clock signal (**CLK**) to the LOAD signal
 - On each clock cycle, load Input value

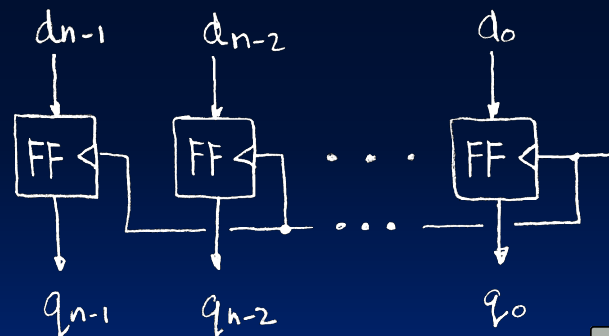
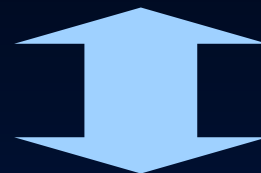
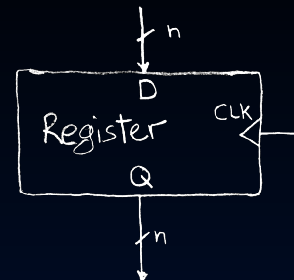


The “triangle” symbol on a block implies a **clocked state element**.

Yan, SP26

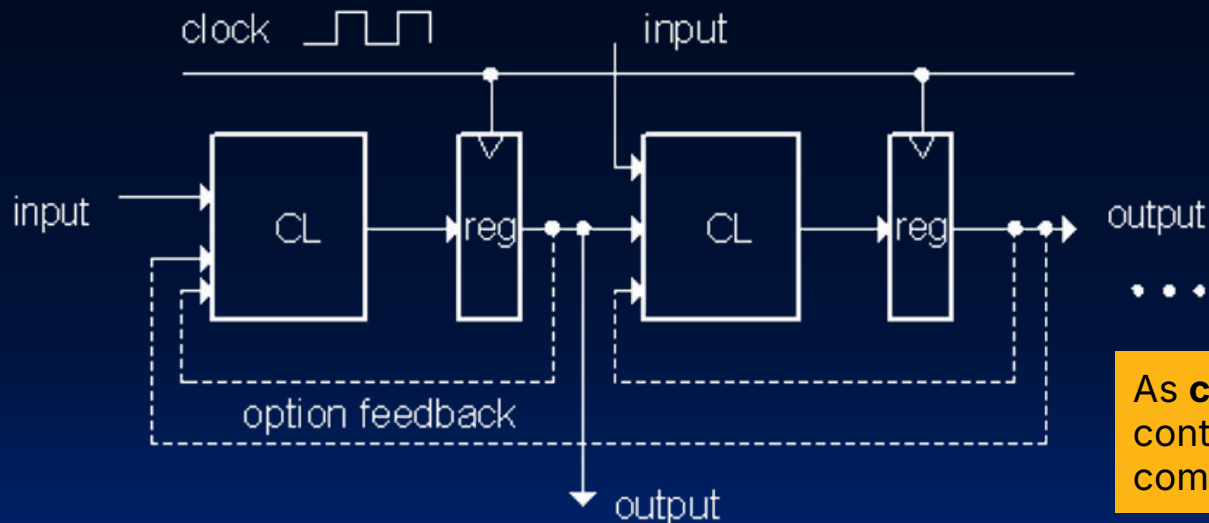
What's Inside a Register?

- Conventional register design:
 - D: Data
 - Q: Output
 - CLK: signal
- An **n-bit wide register** is a parallel collection of **n flip-flops**:
 - Flip-flop (FF): 1-bit register (named as such because the output "flips and flops" between 0 and 1).



General Model for Synchronous Systems

- Collection of Combinational Logic (CL blocks) separated by registers.
 - Registers may be back-to-back; CL blocks may be back-to-back.
- Feedback is optional.
- Clock signal(s) connects only to clock input of registers.



As **clocked elements**, registers control flow of information between combinational logic blocks.

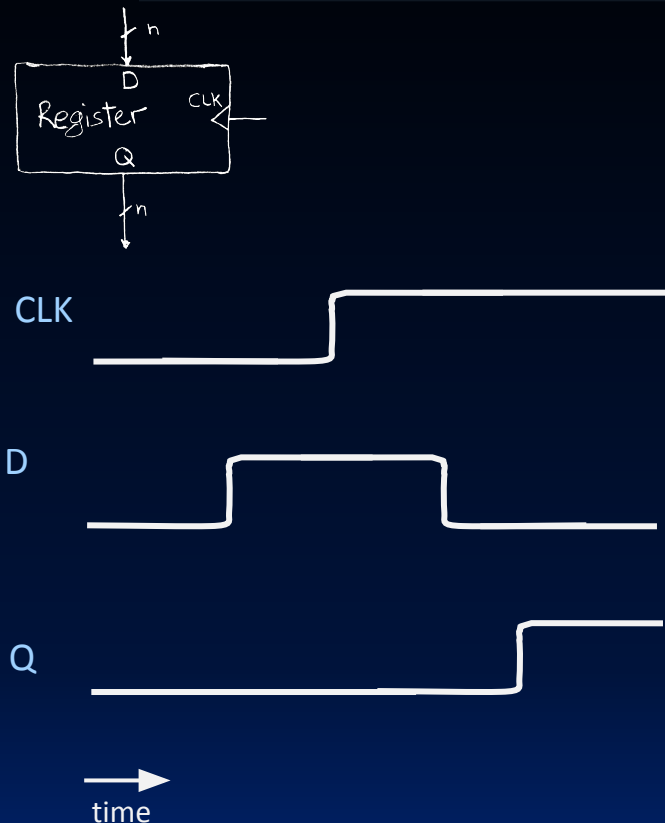


Register Timing

Because the positive edge-triggered register is a **physical system**, there are timing delays in updating the output.

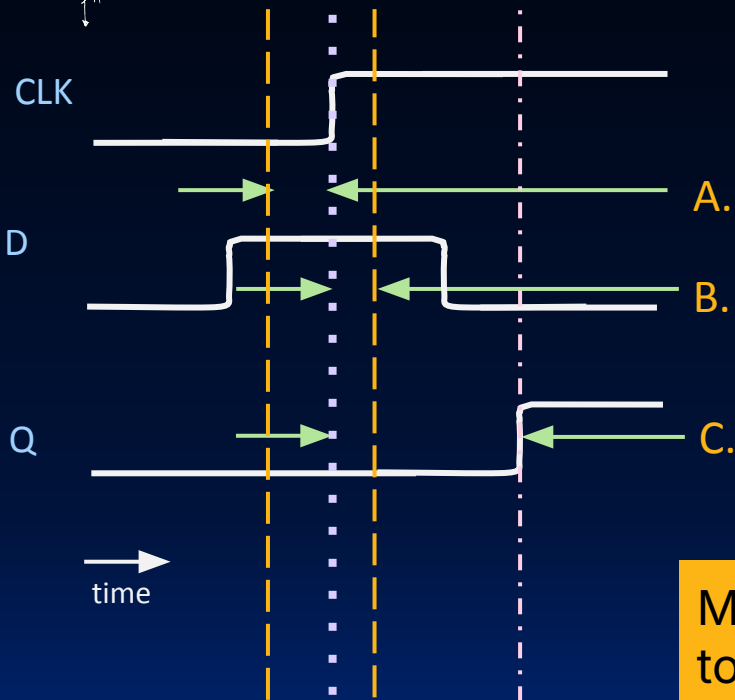
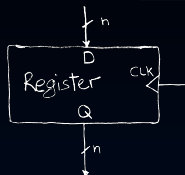
- Clock and Signals
- Registers
- Register Timing
- Circuit Timing Constraints
- Finite State Machines

Register Timing: Terminology



- On the **rising edge** of CLK (signal), each flip-flop **captures the input value d**, stores it internally, and **transfers it to output q**.
- As physical components, flip-flops (and therefore registers) have timing constraints:
 - Setup time**
 - Hold time**
 - CLK-to-Q delay**

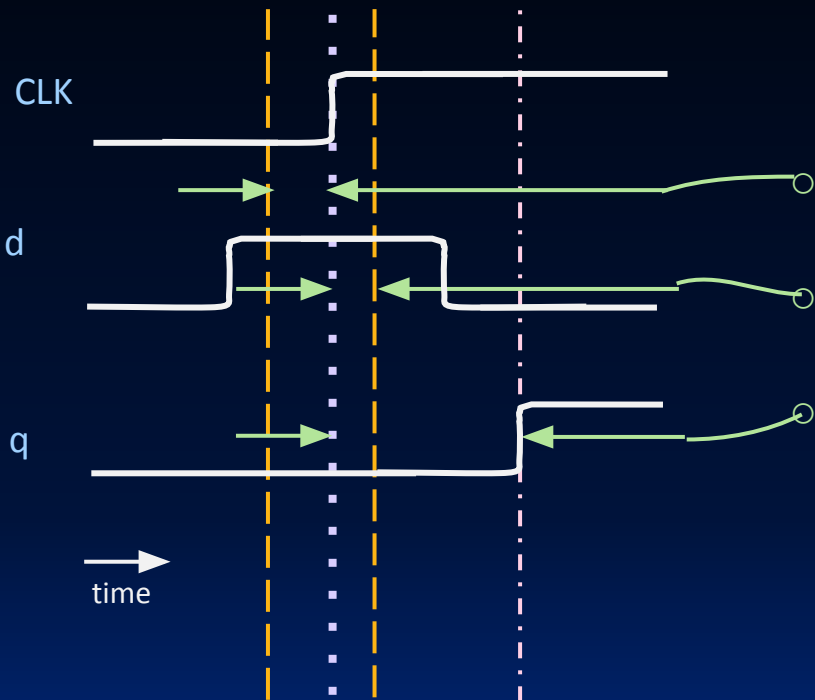
Flip-Flop Timing: Matching



- **Setup time:** How long input must be stable before the rising edge of the clock.
- **Hold time:** How long the input must be stable after the rising edge of the clock.
- **CLK-to-Q delay:** How long it takes the output to change, as measured from the rising edge of the CLK

Match the different timing constraints to A, B, C.

Flip-Flop Timing: Matching

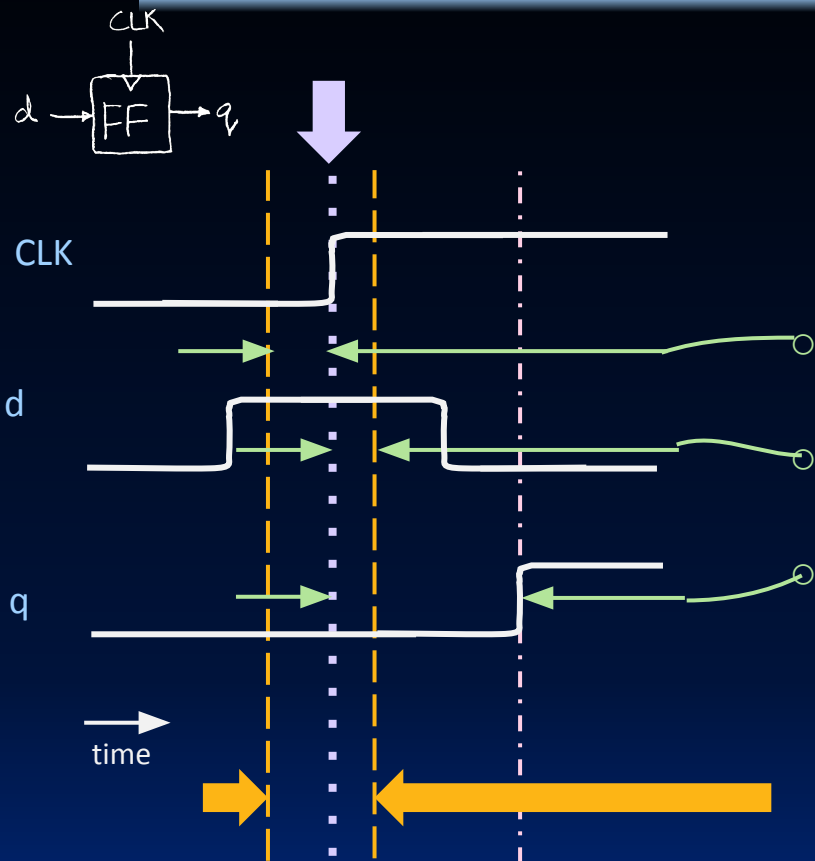


Setup time: How long input must be stable before the rising edge of the clock.

Hold time: How long the input must be stable after the rising edge of the clock.

CLK-to-Q delay: How long it takes the output to change, as measured from the rising edge of the CLK

Flip-Flop Timing: Notes



All register timing constraints are with respect to **rising edge** (of CLK)!

Setup time

Hold time

CLK-to-Q delay

Input data must be stable during setup + hold period!



Circuit Timing Constraints

- Clock and Signals
- Registers
- Register Timing
- Circuit Timing Constraints
- Finite State Machines



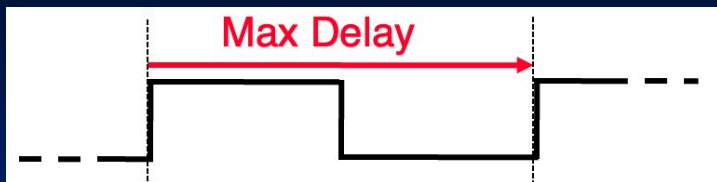
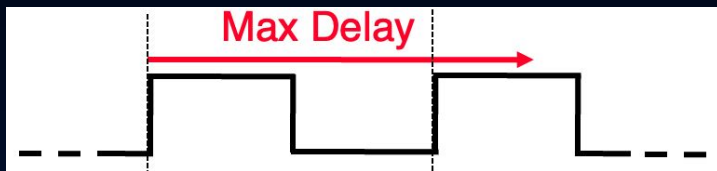
High-bandwidth circuitry

- We seek to create high-bandwidth circuits:
 - Goal: produce many outputs per second
 - As the **clock runs faster** (higher **clock frequency**), state elements change faster.
- **Clock period:** T in seconds (or ns = 10^{-9} s)
- **Clock frequency:** $f = 1/T$, in Hz (or GHz = 1 billion cycles/sec)
- A circuit that runs too fast can cause unstable outputs.

Goal: Determine the **maximum clock frequency** (equivalently, **minimum clock period**) at which to run a circuit.

How does the clock period relate to the Max Delay?

- **Clock period:** T in seconds (or ns = 10^{-9} s)
- **Clock frequency:** $f = 1/T$, in Hz (or GHz = 1 billion cycles/sec)

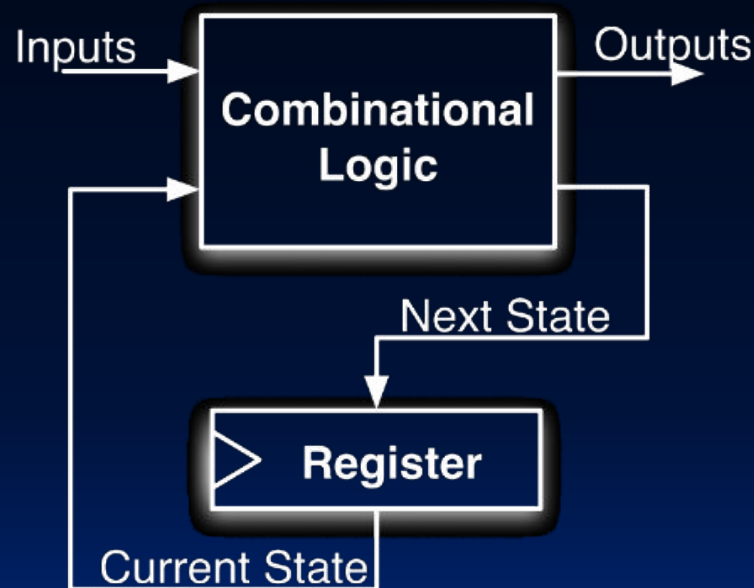


- If the clock period is shorter than the maximum circuit delay, the circuit will not function correctly!
- If the clock period is longer than the max delay. This is wasteful, as the clock period can be shortened.
- Now the clock period is minimized. **Minimum period means maximum frequency!**

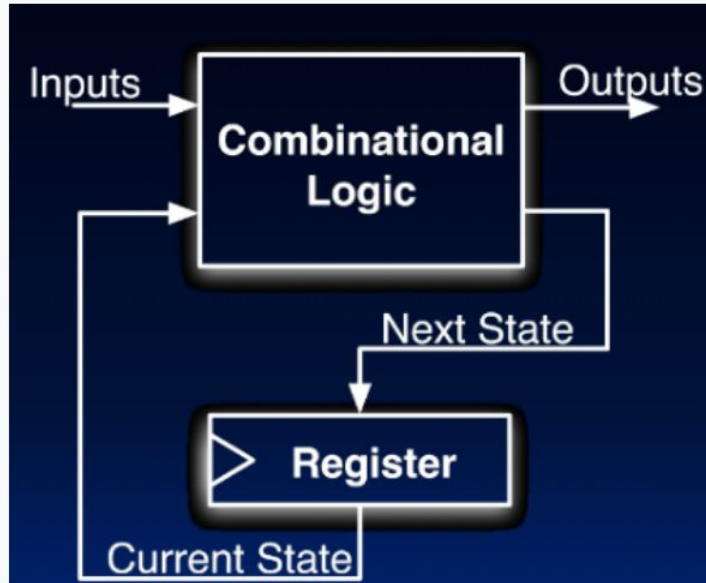
What is the Minimum Clock Period?

The minimum clock period is a sum of the following components (select all that apply):

- A. Setup time
- B. Hold time
- C. CLK-to-Q delay
- D. Propagation delay of combinational logic block
- E. Something else



L21: The minimum clock period is a sum of the following components (select all that apply):



A. Setup time

0%

B. Hold time

0%

C. CLK-to-Q delay

0%

D. Propagation delay of combinational logic block

0%

E. Something else

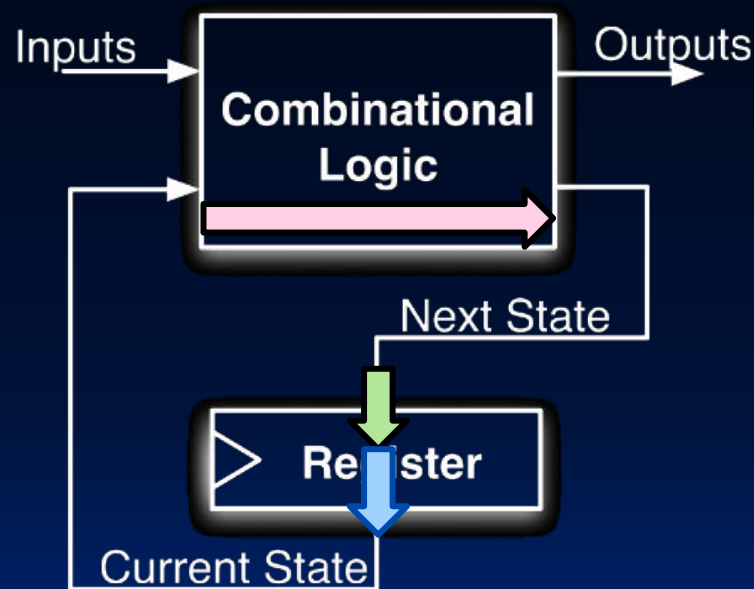
0%



Critical Path Determines Maximum Clock Frequency

- The minimum clock period is the delay on the **critical path**.
 - **Critical path:** The path between input(s) and output(s) that incurs worst-case (maximum) delay.
 - Run the clock any faster, and your circuit becomes unstable!

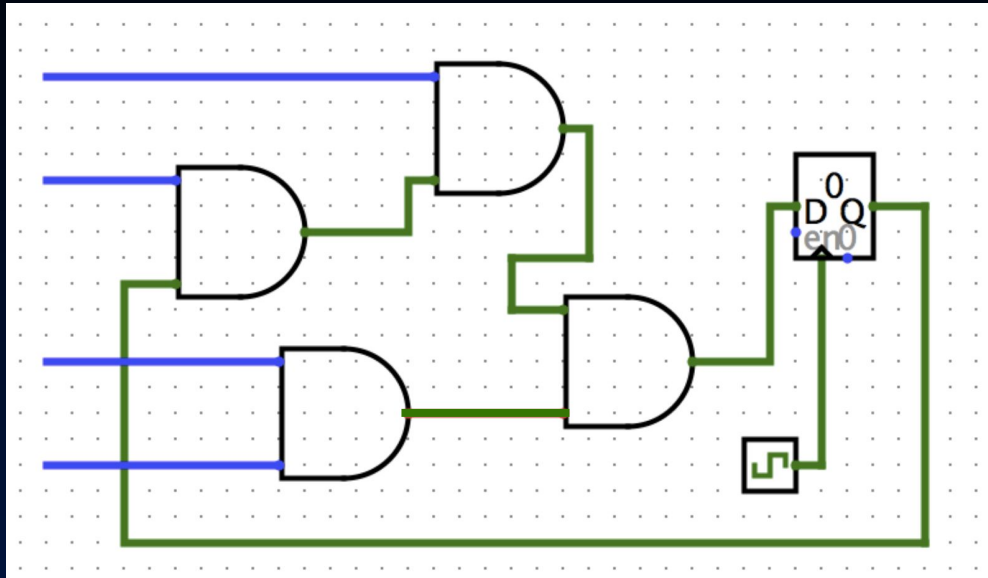
Max Delay of this Circuit =
Delay of Critical Path =
CLK-to-Q Delay
+ **CL Delay**
+ **Setup Time**



Max Clock Frequency Practice

What is the maximum clock frequency for this circuit?

- A. 5 GHz
- B. 200 MHz
- C. 500 MHz
- D. 1/7 GHz
- E. 1/6 GHz
- F. Something else



clk-to-q delay: 1ns

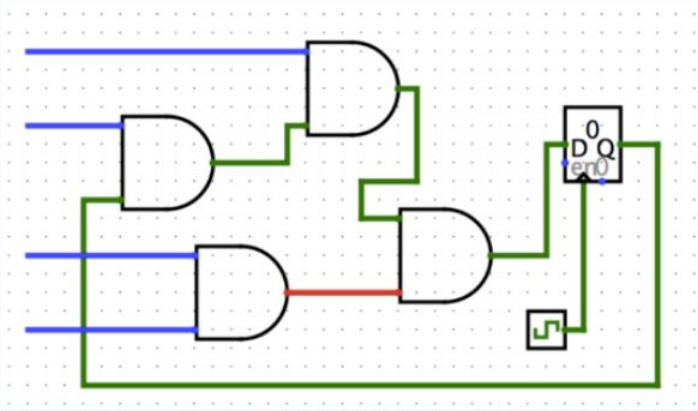
setup time: 1ns

hold time: 1ns

AND delay: 1ns

(Assume all unconnected inputs
come from some other registers.)

L21: What is the maximum clock frequency for this circuit?



clk-to-q delay: 1ns hold time: 1ns
setup time: 1ns AND delay: 1ns
(Assume all unconnected inputs
come from some register.)

A. 5 GHz

0%

B. 200 MHz

0%

C. 500 MHz

0%

D. 1/7 GHz

0%

E. 1/6 GHz

0%

F. Something else

0%



What is the maximum clock frequency for this circuit?

-

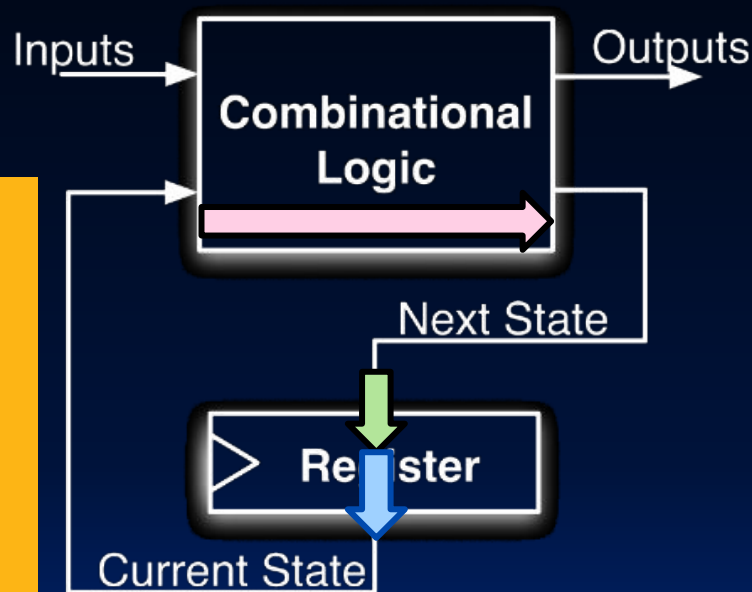
(Assume all unconnected inputs come from some other registers.)

Hold Time: A Second Timing Constraint

- Hold-Time Violation: an alternate problem
 - If CLK-to-Q + **best-case** combinational delay < **Hold Time**...
 - Then data propagates...
 - ...and now you don't hold input long enough!

Circuit should additionally satisfy this constraint:

$$\begin{aligned} &\text{CLK-to-Q} \\ &+ \text{best-case} \\ &\quad \text{combinational delay} \\ &\geq \text{hold time} \end{aligned}$$



Summary: Timing Constraints

1. Set clock period to be (at minimum) the delay through the critical path.
 - **Critical path:** path between input/output that incurs max delay
 - CLK-to-Q + worst-case combinational logic delay + setup time
 - Set maximum clock frequency = $1/(\text{minimum clock period})$
2. Adjust combinational logic to avoid hold-time violations.
 - **Hold time violation:** CLK-to-Q + best case combinational logic delay is faster than ($<$) register hold time
 - Add arbitrary delay to combinational logic to adjust such that CLK-to-Q + best-case combinational delay $\geq$ hold time

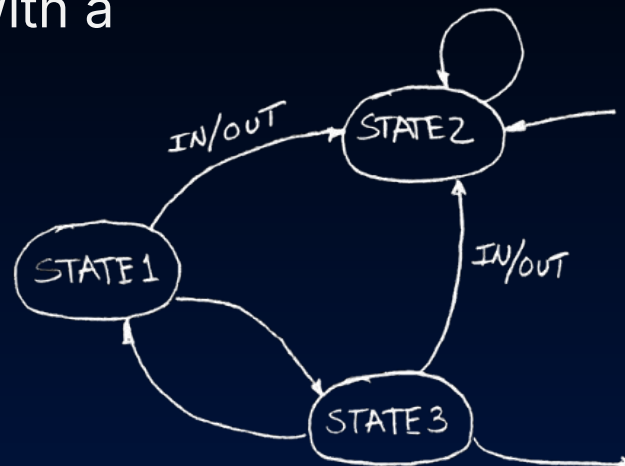


Finite State Machines

- Clock and Signals
- Registers
- Register Timing
- Circuit Timing Constraints
- Finite State Machines

Finite State Machines (FSM) Introduction

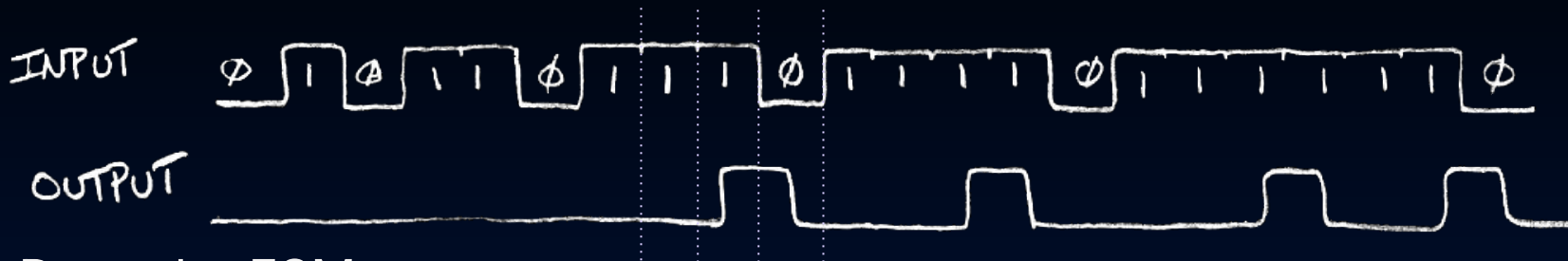
- You may have seen FSMs in other classes.
 - Same basic idea
- The function can be represented with a "**state transition diagram.**"
 - Terminology on diagram



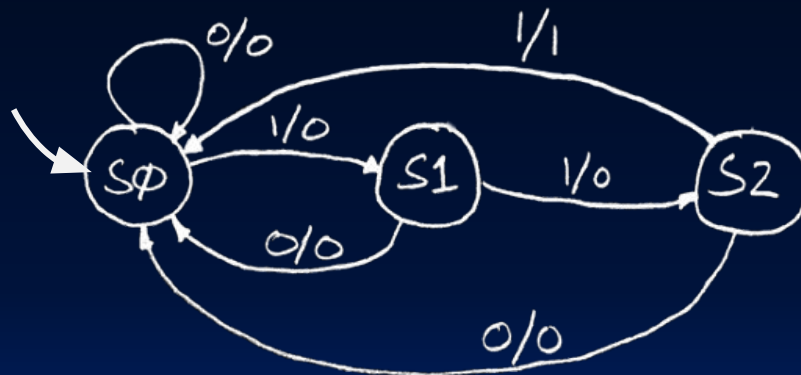
Any FSM can be implemented in hardware as combinational logic and registers.

FSM Example: 3 ones

- FSM to detect the occurrence of 3 consecutive 1's in the input.



- Draw the FSM:
 - Assume state transitions are controlled by the clock.
 - On each clock cycle** the machine checks the inputs
 - Then moves to a new state and produces a new output

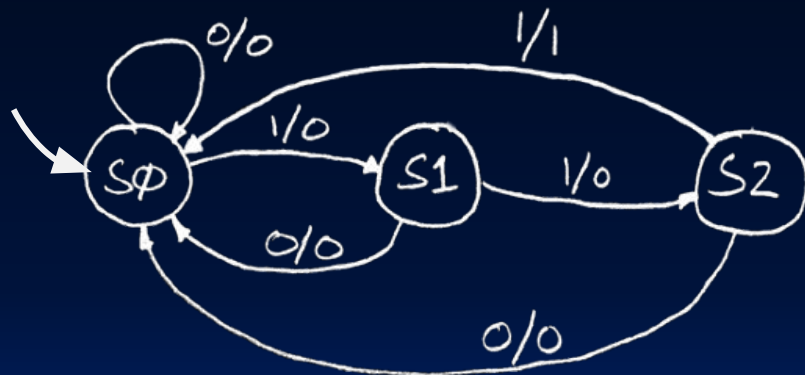


Hardware Implementation of FSM (1/2)

PS	input	NS	output
00	0	00	0
00	1	01	0
01	0	00	0
01	1	10	0
10	0	00	0
10	1	00	1

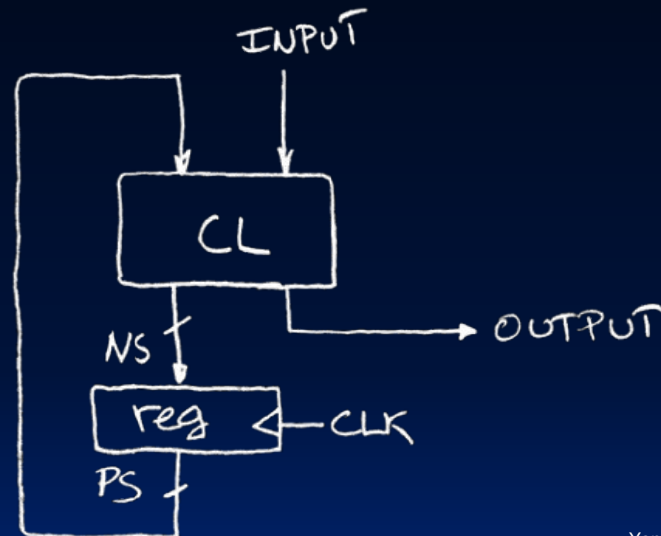
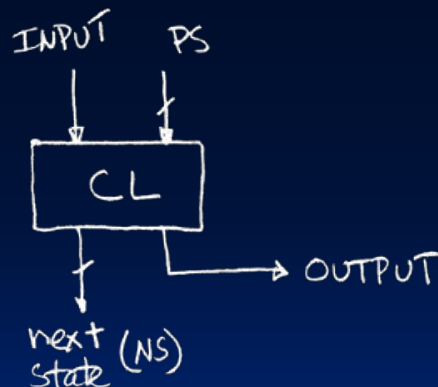
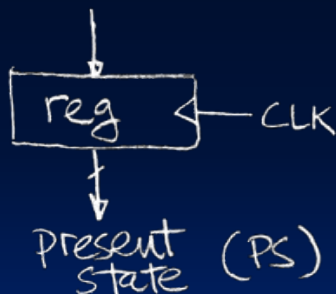
Each row of the truth table is one edge/transition.

With combinational logic and registers, any FSM can be implemented in hardware.



Hardware Implementation of FSM (2/2)

- A **register** is needed to remember which state the machine is in.
 - Use a **unique bit pattern** for each state.
- **Combinational logic circuit** implements a function that maps:
 - the input and present state (PS)
 - to the next state (NS) and output.





"And in conclusion..."

- State elements are used to:
 - Build memories
 - Control the flow of information between other state elements and combinational logic
- D-flip-flops used to build registers
- Clocks tell us when D-flip-flops change
 - setup and hold times are important
- Finite state machines extremely useful
 - You'll see them again 151A, 152, 164, 172, ...
- [later] We pipeline long-delay CL for faster clock